

TaskFlow - Session 4B Project Guide

Understand it. Clean it. Type it.

Today we set TypeScript aside for a while. We reviewed the whole application again in plain JavaScript, cleaned it up a little, saw a real reason TypeScript is worth having, and then rebuilt its types - this time with TypeScript checking the code more carefully than ever before.

Keep this guide. It describes **Session 4B End** exactly as the real project files exist.

1. How HTML, CSS, and TypeScript Fit Together

- `index.html` and `css/style.css` - unchanged in structure from earlier sessions.
- `ts/app.ts` - the TypeScript source we write and edit.
- `js/app.js` - generated automatically by `tsc`. Never edited by hand.
- `tsconfig.json` - tells `tsc` how to compile the project, and which rules to check.

```
ts/app.ts    -->    tsc    -->    js/app.js    -->    browser
(we write this)  (checks types)  (browser runs this)
```

2. Why We Reviewed the JavaScript First

Before touching types again, we read the whole application - HTML, CSS, and JavaScript - and asked whether the JavaScript could be clearer. Two real improvements came out of that:

One function instead of two, doing the same count:

```
function updateTaskSummary(): void {
    let completed: number = 0;

    for (const task of tasks) {
        if (task.completed) {
            completed++;
        }
    }

    const pending = tasks.length - completed;
```

```

    totalCount.textContent = String(tasks.length);
    completedCount.textContent = String(completed);
    pendingCount.textContent = String(pending);

    progressText.textContent = `${completed} of ${tasks.length} tasks
completed`;
}

```

This replaced two separate functions that each counted completed tasks on their own.

A calculation moved outside a loop:

```

function getVisibleTasks(): Task[] {
    const visibleTasks: Task[] = [];
    const search = searchText.toLowerCase();

    for (const task of tasks) {
        // ...
        const title: string = task.title.toLowerCase();
        const matchesSearch: boolean = title.includes(search);
        // ...
    }

    return visibleTasks;
}

```

`searchText.toLowerCase()` doesn't change while the loop runs, so it's calculated once, before the loop starts, instead of on every task.

Neither change altered what the app does - only how the code gets there.

3. Why TypeScript Came Back

Before rebuilding the types, we saw a real problem plain JavaScript couldn't catch. If a function meant to receive a person's `id` (a number) is accidentally given a text version of it instead (`"2"` instead of `2`), a comparison like `task.userId === selectedUserId` silently fails - `2 === "2"` is `false` in JavaScript, because strict equality never treats a number and a string as the same value.

JavaScript never warned us. The browser never warned us. The only sign anything was wrong was the person filter quietly not working. TypeScript can catch exactly this kind of mistake, before the app ever runs, once a function's parameter has a real type.

4. Task and User Interfaces

```
interface Task {
  userId: number;
  title: string;
  completed: boolean;
}

interface User {
  id: number;
  name: string;
}
```

An `interface` describes the shape of an object - which properties it has, and what type each one is. Our `Task` interface doesn't name `id`, even though the real API sends one, because nothing in this app reads `task.id`. An interface only needs to describe what the code actually uses.

5. Useful Explicit Types We Keep

```
let tasks: Task[] = [];
let users: User[] = [];
```

An empty array `[]` could hold anything - without `Task[] / User[]`, TypeScript would have nothing to check against.

```
let currentFilter: string = "all";
let searchText: string = "";
let selectedUserId: number = 0;
```

These are useful to write explicitly - each one names exactly what kind of value that piece of state holds.

```
function getUsername(userId: number): string {
```

A parameter type and a return type are promises: this function will always be given a number, and will always hand back a string.

6. Type Inference - When We Don't Repeat a Type

```
BEFORE: let completed: number = 0;
AFTER:  let completed = 0;

BEFORE: const matchesSearch: boolean = title.includes(search);
AFTER:  const matchesSearch = title.includes(search);
```

If TypeScript can already work out an obvious local type from the value on the right of `=`, we don't always need to repeat it. This is different from Section 5 above - those types are kept because TypeScript has *nothing* to infer from (an empty array) or because they're a promise about a function's contract (a parameter or return type).

7. `document.querySelector()` and `null`

`document.querySelector()` can genuinely return `null` at runtime, if nothing on the page matches the id - this is true no matter what any compiler setting says. One place in the file checks for it, as good practice:

```
function renderTasks(): void {
    if (!taskList) {
        return;
    }

    // ...taskList can be used safely below this line...
}
```

The search box needs a more specific type than plain `Element`, because `.value` only exists on input elements:

```
const searchInput = document.querySelector("#searchInput") as HTMLInputElement
| null;
```

8. Five Compiler Rules Are Now Active

By the official end of this session, `tsconfig.json` has five stricter settings turned on, all at once:

```
{
  "compilerOptions": {
    "noImplicitAny": true,
    "strictNullChecks": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noImplicitReturns": true
  }
}
```

Rule	What it checks
<code>noImplicitAny</code>	Every function parameter must have a real type - TypeScript won't silently allow "could be anything."
<code>strictNullChecks</code>	Anything that might be <code>null</code> (like a <code>querySelector()</code> result) must be checked before it's used.
<code>noUnusedLocals</code>	A local variable that's created but never read is flagged.
<code>noUnusedParameters</code>	A function parameter that's accepted but never read is flagged.
<code>noImplicitReturns</code>	A function that returns a value on some paths should not finish without returning on another path.

With all five on, functions that use a possibly-`null` DOM element now check it first, the same `if (!x) { return; }` idea already familiar from `renderTasks()` - just applied everywhere it's needed, not just the one place it was demonstrated.

9. What Stayed Exactly the Same

- Every feature: tasks and users loading, owner names, the "Unknown person" fallback, the "Tasks per person" summary, person filtering, "All people," status filters, search, all combined, the empty state, stats, and the progress sentence.
- The visual design - same layout, same colors, same responsive behavior.
- The overall shape of the application - the same functions, doing the same jobs, just reviewed, a little clearer, and checked more carefully.

You Reviewed, Cleaned Up, and Re-Typed TaskFlow

You can now:

- explain what every part of TaskFlow's HTML, CSS, and JavaScript does, without looking anything up,
- recognize and remove real duplication in JavaScript, using only syntax you already knew,
- describe a real situation where plain JavaScript accepts a value of the wrong type silently,
- rebuild interfaces, typed arrays, typed parameters, typed returns, and API assertions from a clean JavaScript file,
- explain what each of five stricter compiler rules checks for, and fix the kind of issue each one finds.

The most valuable thing about today is not any single line of code. It's this:

Code you already trust is still worth reading again. Understanding it, cleaning it, and then asking a stricter tool to check it more carefully - in that order - is a normal, healthy way to come back to a project, not a sign something was wrong with it.